

SMOHIT ARMAN SWAIN

192572092

PYTHON PROGRAMMING

SCENARIO BASED PROBLEM

Student 33   **Smohit Arman Swain**   192572092RC

4 / 4 submitted

**Q1. Intelligent Customer Feedback Analysis**

CODE

```
def preprocess(text):
    text = text.lower()
    text = text.strip()
    text = text.replace(".", "")
    text = text.replace(",", "")
    text = text.replace("!", "")
    text = text.replace("?", "")
    return text

def analyze(feedback):
    positive = ["good", "excellent", "great", "happy", "best", "nice", "love"]
    negative = ["bad", "poor", "worst", "hate", "slow", "problem", "terrible"]
    pos = 0
    neg = 0
    words = feedback.split()
    for word in words:
        if word in positive:
            pos += 1
        elif word in negative:
            neg += 1
    return pos, neg

def classify(pos, neg):
    if pos > neg:
        return "Positive"
    elif neg > pos:
        return "Negative"
    else:
        return "Neutral"

# Main Program
n = int(input())
for i in range(n):
    feedback = input()
    feedback = preprocess(feedback)
    pos, neg = analyze(feedback)
    result = classify(pos, neg)
    print("\nProcessed Feedback:", feedback)
    print("Positive Words:", pos)
    print("Negative Words:", neg)
    print("Category:", result)
```

INPUT

```
3
Enter feedback: The product is excellent and good.
Enter feedback: The service is very bad and slow.
Enter feedback: Product is good but delivery is slow.
```

OUTPUT

```
Processed Feedback: enter feedback: the product is excellent and good
Positive Words: 2
Negative Words: 0
Category: Positive

Processed Feedback: enter feedback: the service is very bad and slow
Positive Words: 0
Negative Words: 2
Category: Negative

Processed Feedback: enter feedback: product is good but delivery is slow
Positive Words: 1
Negative Words: 1
Category: Neutral
```

1.

## Q2. Comprehensive Password Strength Checker

CODE

```
def analyze_password(password):
    upper = 0
    lower = 0
    digit = 0
    symbol = 0
    special = "!@#$%^&*()-_+[]{}|\\:;'\",./'~"
    # Character-level analysis
    for ch in password:
        if ch.isupper():
            upper += 1
        elif ch.islower():
            lower += 1
        elif ch.isdigit():
            digit += 1
        elif ch in special:
            symbol += 1
    return upper, lower, digit, symbol

def check_patterns(password):
    # Common weak passwords
    weak_list = [
        "password", "123456", "12345678",
        "admin", "qwerty", "abc123"
    ]
    if password.lower() in weak_list:
        return "Common password"
    # All characters same
    if len(password) > 0 and password == password[0] * len(password):
        return "Repeated characters"
    # Sequential numbers
    if password in ["123456", "12345678", "987654321"]:
        return "Sequential numbers"
    return "None"

def password_strength(password):
    if len(password) == 0:
        print("Password cannot be empty.")
        return
    if len(password) > 50:
        print("Very long password detected.")
    upper, lower, digit, symbol = analyze_password(password)
    score = 0
    if len(password) ≥ 8:
        score += 1
    if upper > 0:
        score += 1
    if lower > 0:
        score += 1
```

2.

```

pattern = check_patterns(password)
if pattern != "None":
    score -= 2
# Classification
if score <= 2:
    strength = "Weak"
elif score == 3:
    strength = "Moderate"
elif score == 4:
    strength = "Strong"
else:
    strength = "Very Strong"
print("\n—— Password Analysis ——")
print("Password Length :", len(password))
print("Uppercase Letters :", upper)
print("Lowercase Letters :", lower)
print("Digits :", digit)

```

simatslab.saveetha.com/practice\_app/classactivity/ajax/download\_submissions.php

109/126

7/29/26, 8:35 PM

Student Submissions — Scenario Based Program Writing

**SIMATS Engineering** • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

```

print("Special Symbols :", symbol)
print("Pattern Check :", pattern)
print("Strength :", strength)
print("\nSuggestions:")
if len(password) < 8:
    print("- Use at least 8 characters.")
if upper == 0:
    print("- Add uppercase letters.")
if lower == 0:
    print("- Add lowercase letters.")
if digit == 0:
    print("- Add digits.")
if symbol == 0:
    print("- Add special symbols.")
if pattern != "None":
    print("- Avoid common or predictable passwords.")
# Main Program
pwd = input()
password_strength(pwd)

```

INPUT Hello@123

OUTPUT (no output)

### Q3. Structured Server Log Error Analyser

CODE

```
def extract_errors(logs):
    """Filters log entries to find and return only error-related lines."""
    errors = []
    for line in logs:
        # Using string searching 'in' to identify relevant lines
        if "ERROR" in line:
            errors.append(line.strip())
    return errors

def remove_duplicates(error_list):
    """Removes duplicate error messages while preserving order."""
    unique_errors = []
    for error in error_list:
        if error not in unique_errors:
            unique_errors.append(error)
    return unique_errors

def categorize_and_count_errors(error_list):
    """Counts and categorizes errors using string searching methods."""
    summary = {
        "Database Errors": 0,
        "Network Errors": 0,
        "Timeout Errors": 0,
        "Other Errors": 0
    }

    for error in error_list:
        # Using find() and startswith() to demonstrate required skills
        if "DB_" in error or error.lower().find("database") != -1:
            summary["Database Errors"] += 1
        elif error.startswith("NET_") or "network" in error.lower():
            summary["Network Errors"] += 1
        elif "timeout" in error.lower():
            summary["Timeout Errors"] += 1
        else:
            summary["Other Errors"] += 1

    return summary
```

3.

```
def display_report(unique_errors, summary):
    """Prints a concise report of unique errors and their summaries."""
    print("--- Unique Error Logs ---")
    for error in unique_errors:
        print(error)

    print("\n--- Error Summary Report ---")
    for error_type, count in summary.items():
        print(f"{error_type}: {count}")

# --- Sample Execution ---
if __name__ == "__main__":
    # Simulated server log input
    sample_logs = [
        "INFO: System initialized successfully.",
        "ERROR: DB_CONNECTION_FAILED - Cannot connect to database instance.",
        "WARNING: High memory usage detected.",
        "ERROR: NET_TIMEOUT - Connection timed out while reaching API.",
        "ERROR: DB_CONNECTION_FAILED - Cannot connect to database instance.", # Duplicate
        "ERROR: UNKNOWN_EXCEPTION - Something unexpected occurred.",
        "INFO: Periodic cleanup task completed.",
        "ERROR: NET_TIMEOUT - Connection timed out while reaching API." # Duplicate
    ]
```

simatslab.saveetha.com/practice\_app/classactivity/ajax/download\_submissions.php

111/126

7/29/26, 8:35 PM

Student Submissions — Scenario Based Program Writing

**SIMATS Engineering** • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

```
# 1. Extraction
all_errors = extract_errors(sample_logs)

# 2. Duplicate Removal
cleaned_errors = remove_duplicates(all_errors)

# 3. Categorization & Counting (Analyzes the complete list of occurrences)
error_counts = categorize_and_count_errors(all_errors)

# 4. Final Reporting
display_report(cleaned_errors, error_counts)
```

OUTPUT (no output)

#### Q4. Email Domain Frequency Analyser

CODE

```
def extract_domains(emails):
    domains = []

    for email in emails:
        if "@" in email:
            parts = email.split("@")

            if len(parts) == 2 and parts[0] != "" and parts[1] != "":
                domains.append(parts[1])
            else:
                print(email, "→ Invalid Email")
        else:
            print(email, "→ Invalid Email")

    return domains

def count_domains(domains):
    count = {}

    for domain in domains:
        if domain in count:
            count[domain] += 1
        else:
            count[domain] = 1

    return count

def most_common(count):
    max_count = 0
    max_domain = ""

    for domain in count:
        if count[domain] > max_count:
            max_count = count[domain]
            max_domain = domain

    return max_domain, max_count
```

4.

```
# Main Program
n = int(input("Enter number of email addresses: "))

emails = []

for i in range(n):
    email = input("Enter email: ")
    emails.append(email)

domains = extract_domains(emails)
counts = count_domains(domains)

print("\nDomain Frequencies")
for domain in counts:
    print(domain, ":", counts[domain])

domain, freq = most_common(counts)

if domain != "":
    print("\nMost Common Domain:", domain)
    print("Frequency:", freq)
else:
    print("\nNo valid email addresses found.")
```

INPUT

```
5
abc@gmail.com
```

[simatslab.saveetha.com/practice\\_app/classactivity/ajax/download\\_submissions.php](https://simatslab.saveetha.com/practice_app/classactivity/ajax/download_submissions.php)

113/126

7/29/26, 8:35 PM

Student Submissions — Scenario Based Program Writing

**SIMATS Engineering** • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

```
xyz@gmail.com
test@gmail.com
hello@outlook.com
ram@yahoo.com
```

OUTPUT

```
Enter number of email addresses: Enter email: Enter email: Enter email: Enter email: Enter email:
Domain Frequencies
gmail.com : 3
outlook.com : 1
yahoo.com : 1

Most Common Domain: gmail.com
Frequency: 3
```